

Reviewer Pack — Minimal Number of Modular Forms and Resolution Proof Width I...

Entry 3104545bd127 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-04 02:31:58 UTC

Minimal Number of Modular Forms and Resolution Proof Width Inequality

Entry ID: 3104545bd127 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-04 02:31:58 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Number Theory (Modular Forms) **Field B** (complexity object): Resolution Proof Complexity

Statement:

The resolution proof width $w(\varphi)$ of a Tseitin formula φ with n variables is upper-bounded by the number N of distinct Hecke eigenvalues for the Hecke eigenform associated with φ , such that $w(\varphi) = O(N^{(1/2)})$.

Rationale (proposer's reasoning):

Modular forms provide a rich set of arithmetic invariants that could potentially encode information about the complexity of computational tasks. The conjecture posits that the eigenvalue structure of modular forms might be related to the geometric complexity of the corresponding Boolean functions, as measured by resolution proof width.

Taxonomy category: TROPICAL_FOURIER_ANALYSIS (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 0266ced2bf8b98f2

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

To support the conjecture, the correlation coefficient between the resolution proof width $w(\varphi)$ of Tseitin formulas φ and the number N of distinct Hecke eigenvalues must be greater than or equal to 0.8 for all seeds, with a mean metric value less than or equal to 3.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.90	SAFE	UNCERTAIN
KARP_LIPTON	SAFE	1.00	SAFE	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "algebraic number theory" AND modular forms AND resolution proof complexity" - "Hecke eigenforms" AND Tseitin formula AND resolution proof width" - "resolution proof width" AND upper bound Hecke eigenvalues modular forms

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_tseitin_formula(n):
        variables = [f'x{i}' for i in range(1, n+1)]
        clauses = []

        # Generate clauses for each variable
        for var in variables:
            clause = [var]
            neg_clause = [-var]
            clauses.append(clause)
            clauses.append(neg_clause)

        # Generate clauses for implications
        for i in range(1, n):
            for j in range(i+1, n+1):
                pos_ij = f'x{i} x{j}'
                neg_ij = [-i, -j]
                clauses.append([pos_ij])
                clauses.append(neg_ij)

        return clauses

    def resolution(clauses):
        clauses = [set(c) for c in clauses]
        new_clauses = set()
        while True:
            new_clauses.clear()
```

```

    for clause1 in clauses:
        for clause2 in clauses:
            if len(clause1 & clause2) == 1:
                new_clause = (clause1 | clause2) - {list(clause1 & clause2)[0]}
                if not new_clause:
                    return None
                new_clauses.add(frozenset(new_clause))
        if new_clauses.issubset(clauses):
            break
        clauses.update(new_clauses)
    return len(clauses)

def hecke_eigenvalues(clauses):
    n = len(clauses[0])
    vector = [0] * ((n + 1) // 2)

    for clause in clauses:
        if len(clause) == 1:
            i = abs(int(clause.pop()))
            if i % 2 == 0:
                vector[(i-1)//2] = 1
            else:
                vector[(i+1)//2] = -1

    N = sum(abs(x) for x in vector)
    return N

n_values = [5, 10, 15, 20, 30, 40]
metric_values = []

for n in n_values:
    clauses = generate_tseitin_formula(n)
    width = resolution(clauses)
    if width is None:
        continue
    N = hecke_eigenvalues(clauses)
    if N == 0:
        continue
    metric_value = width / math.sqrt(N)
    metric_values.append(metric_value)

mean_metric_value = sum(metric_values) / len(metric_values)
conjecture_holds = all(m <= 3 for m in metric_values)
counterexample = "" if conjecture_holds else f"correlation_coefficient={mean_metric_value}"

return {
    "metric_name": "resolution_proof_width",
    "metric_value": mean_metric_value,
    "instances_tested": len(metric_values),
    "n_max": max(n_values),
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys

```

```

seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

results = []
for seed in seeds:
    result = run_trial(seed)
    print(f"TRIAL: {result}")
    results.append(result)

mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0 support_fraction={support_fraction}")
elif any(not r["conjecture_holds"] for r in results) and support_fraction >= 0.8:
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample={result['counterexample']} first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE insufficient_support")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_81542629.py", line 107, in <module>
    result = run_trial(seed)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_81542629.py", line 78, in run_trial
    clauses = generate_tseitin_formula(n)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_81542629.py", line 28, in generate_tseitin_formula
    neg_clause = [-var]
                  ^^^
TypeError: bad operand type for unary -: 'str'

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means that it was unable to complete the computation and verify the conjecture. | next: Investigate the cause of the crash in the test code and attempt to run the test again to obtain results.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13434
2	preregistration	ollama_remote	glm4:latest	0	0	11066
3	novelty	ollama_remote	glm4:latest	0	0	27764
4	novelty	ollama_remote	glm4:latest	0	0	24128
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	21769
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	25460
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18007
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	24851
9	judge	ollama_remote	glm4:latest	0	0	8636

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 175115 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/3104545bd127.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/3104545bd127.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/3104545bd127.tar.gz` (if generated)